



Concrete Architecture of Gemini CLI

OMMKPN NotFound 404 Error

Ming Yuan (Team Lead)	–	22xb63@queensu.ca
Michael Liu (Presenter)	–	22jdn2@queensu.ca
Omar Kaddah	–	22chm1@queensu.ca
Enrong Pan (Presenter)	–	enrong.pan@queensu.ca
Nandan Bhut	–	22gp44@queensu.ca
Kosi Amobi-Oleka	–	22fnbn@queensu.ca

Abstract

Gemini CLI is an open-source AI agent developed by Google that brings the capabilities of the Gemini large language model directly into the terminal. This report recovers the concrete architecture of Gemini CLI by analyzing its source code using the Understand static analysis tool[1] and comparing the result against our conceptual architecture from Assignment 1. Based on global feedback[2] and closer examination of the codebase[3], we revised our conceptual model from nine components to seven: the Terminal Interface as the client, and the Orchestration Engine, Toolbox, Extension Manager, Task Delegation System, Security and Execution, and Configuration System on the server side[4]. The recovered concrete architecture largely validates our conceptual design but reveals several unexpected cross-layer dependencies where components are more tightly coupled than anticipated. A reflexion analysis found no absent dependencies but identified numerous divergent ones, which we investigated to determine whether they reflect genuine architectural relationships or refactorable type-level imports. Two use cases, a stock summary retrieval and a file translation delegated to a remote agent, illustrate how the concrete components collaborate across the orchestration, tool execution, and agent delegation pathways.

1 Introduction and Overview

1.1 Overview

Understanding the architecture of a large software system requires examining both its intended design and its actual implementation. In Assignment 1, we developed a conceptual architecture of the Gemini CLI system by analyzing available documentation and identifying the major subsystems, responsibilities, and interactions that define the system at a high level. While this conceptual view provides an important abstraction of the system’s design, it does not necessarily reflect how the system is implemented in practice.

This report focuses on recovering the concrete architecture of Gemini CLI by analyzing its source code and dependency structure. Using the Understand static analysis tool [1], we extracted dependency data and examined relationships among modules in order to identify the system’s major subsystems and the interactions between them. By grouping related components and filtering non-runtime dependencies, we constructed a clearer representation of the architecture as it exists in the implementation.

A key goal of this report is to compare the recovered concrete architecture with the conceptual architecture proposed in Assignment 1. Through this reflexion analysis, we identify similarities and differences between the intended design and the implemented structure, and investigate the reasons for these discrepancies. This comparison provides insight into architectural decisions, implementation constraints, and areas where the system structure diverges from the conceptual model.

The remainder of this report first presents the recovered concrete architecture and its major subsystems, followed by a deeper examination of one selected subsystem and the architectural patterns it exhibits. Finally, we discuss the results of the reflexion analysis and highlight key observations about the design and implementation of the Gemini CLI system.

1.2 Derivation Process

We began by comparing our A1 conceptual architecture against the global feedback[2] and the reference architecture document on onQ. After noting similarities and differences, we consulted Dr. Adams for advice on what to keep and what to change. Based on these discussions, we revised our conceptual architecture: renaming components, merging tightly coupled ones, and adding the Task Delegation System. We also keep the Gemini API Messenger as a dotted-line helper extended from the Orchestration Engine, though we later determined it did not warrant a separate component, as we detail in the subsystem overview. With the updated conceptual architecture as our framework, we loaded the codebase into Understand [1] and began iteratively assigning folders and files to our components. For most directories the names made placement straightforward, for example the tools and policy folders clearly mapped to the Toolbox and Security and Execution respectively. When names were ambiguous, we opened individual files to check what they actually did before deciding.

The process was not smooth. Our first dependency diagram looked far too messy: nearly every component had unexpected bidirectional dependencies, and the Terminal Interface appeared to depend on everything in the system. We dug into the Terminal grouping and discovered that many outgoing dependencies were caused by test files scattered across subfolders, which imported from all over the codebase. After removing all test files, mock directories, and packages unrelated to the runtime architecture, the dependencies became much more reasonable. We also corrected several misplaced files, particularly CLI-side configuration files initially assigned to server-side components, which were creating artificial cross-component dependencies. The remaining divergences were then investigated systematically, which we discuss in the reflexion analysis.

2 Conceptual Architecture

In our first report, we proposed that Gemini CLI adopts a hybrid of two architecture styles: **Client-Server** and **Layered**[4]. The overall system splits into a client side and a server side. On the client side,

the UI component handles all direct user interaction: managing input, displaying output, applying themes, and controlling the terminal experience. On the server side, a Layered architecture divides responsibilities into four layers: the Control Logic Layer (Agent Core Coordinator), the Execution Logic Layer (API Messenger, Memory Manager, Tool Scheduler), the Data Access Layer (Toolbox, Extension Manager), and the Risk Management Layer (Policy Manager, Risk Prevention) [5]. After receiving global feedback from Dr. Adams and discussing it with him in person, we revised our conceptual architecture to better support the concrete architecture recovery.

First, we renamed two components for clarity. The **UI** becomes the **Terminal Interface**, and the **Memory Manager** becomes the **Configuration System**, since this component handles not just conversation context and session history but also configuration loading and GEMINI.md project files. We let the Configuration System span all server-side layers, since every component needs access to it, a “cheating” strategy discussed in lecture.

Second, we merged several components. The **Agent Core Coordinator**, **API Messenger**, and **Tool Scheduler** are collapsed into the **Orchestration Engine**, which is responsible for constructing prompts, communicating with the Gemini model, and coordinating tool execution. Similarly, **Policy Manager** and **Risk Prevention** merge into **Security and Execution**, handling authentication, policy enforcement, and sandboxed execution. While we absorbed the API Messenger into the Orchestration Engine as a merge, we recognize that communication with the remote Gemini API is a distinct responsibility. We therefore show the **Gemini API Messenger** as a dotted-line subcomponent within the Orchestration Engine in Figure 1 and in the sequence diagrams, to indicate that this responsibility exists conceptually even though we do not treat it as a standalone component. The Orchestration Engine handles all API calls itself, but the dotted notation acknowledges this internal boundary.

Third, we added one new component: the **Task Delegation System**. This was identified as a gap in our original report based on the global feedback. The Task Delegation System handles **Agent-to-Agent (A2A)** communication, delegating complex or resource-intensive tasks to remote agents for distributed processing and specialized expertise[5].

Each external interface is contacted by its associated component: the Gemini API by the Orchestration Engine (through its internal API Messenger responsibility), MCP Servers by the Extension Manager, and A2A Servers by the Task Delegation System. To summarize, we renamed two components, merged three into one (with the API Messenger retained as a dotted-line internal subcomponent), merged two into one, and added one new component. The updated conceptual dependency diagram is shown in Figure 1, from which we begin our concrete architecture recovery.

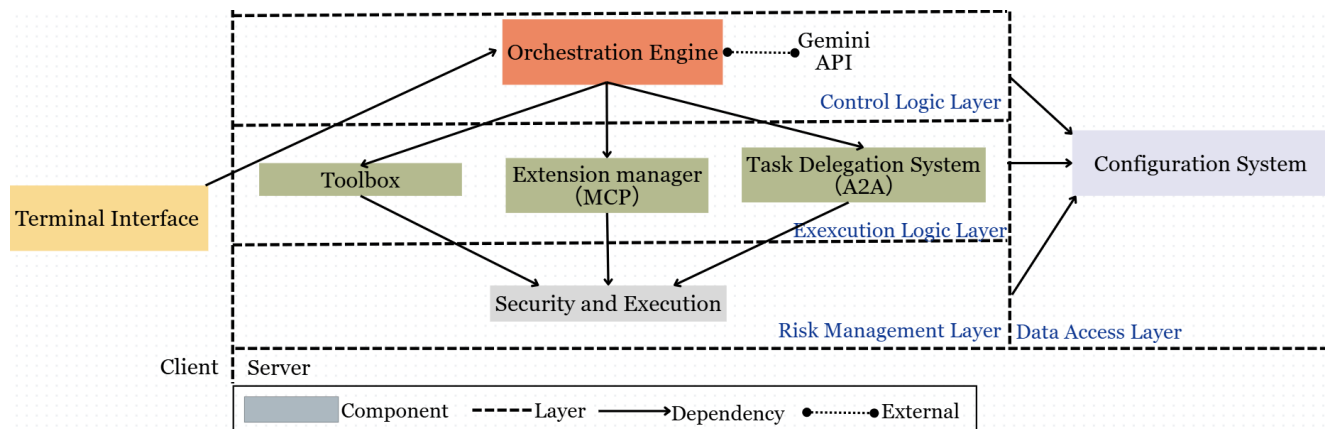


Figure 1: New Conceptual Architecture

3 Concrete Architecture

Since one of the major goals of this report is to recover the concrete architecture of Gemini CLI and compare it with our conceptual architecture proposed earlier, we have decided to use a scitool called **Understand**[1] to help us group the folders, files, and perhaps functions, then visualize the dependencies between components. We started by assuming our conceptual architectural structure, then dragging the folders and files into our proposed components.

3.1 Subsystems Overview

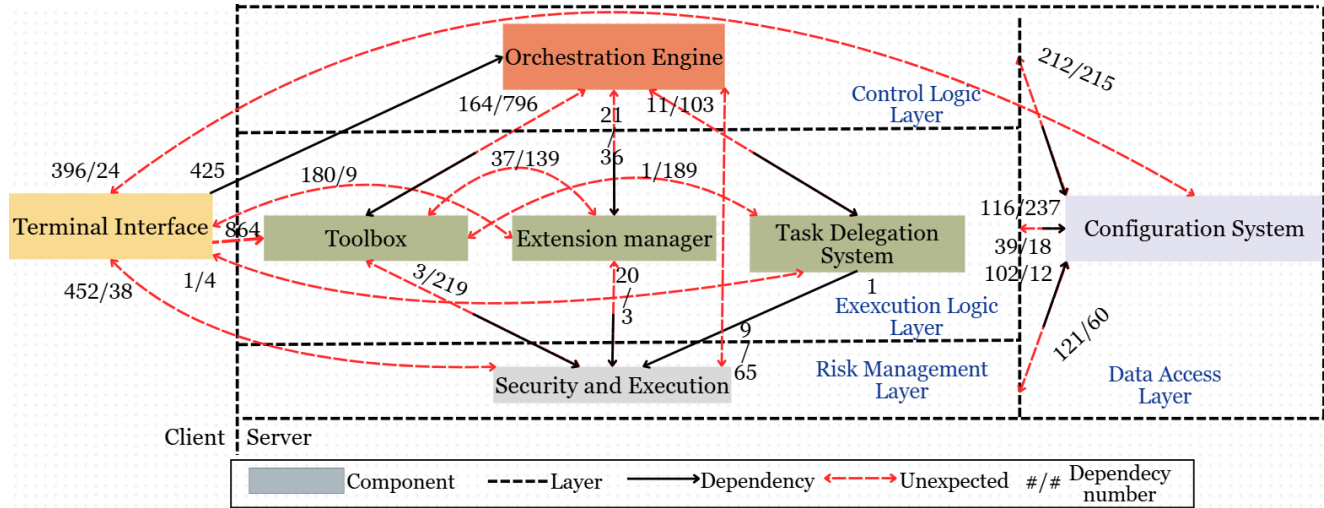


Figure 2: Concrete Architecture

The concrete architecture of Gemini CLI consists of seven components, each mapped to specific directories and files in the codebase using Understand [1]. As noted earlier, the Gemini API Messenger that appeared as a dotted-line subcomponent in our updated conceptual architecture is not represented as a separate component here: after examining the codebase, we found that API communication logic is woven directly into the orchestration and routing modules, sharing state with prompt construction and response handling in a way that makes separation impractical. We therefore treat it as an internal responsibility of the Orchestration Engine. The system still broadly follows the client-server and layered style from our first report. However, digging into the codebase and analyzing dependencies in Understand [1], we found that the system does not strictly follow the hierarchical structure of a layered style. There are multiple cases where non-adjacent layers interact directly, which we discuss in each subsystem description below and further in the reflexion analysis.

3.1.1 Terminal Interface

The Terminal Interface is the client-side component responsible for everything the user directly sees and interacts with. It covers the majority of `packages/cli/`, including the main entry point, interactive and non-interactive CLI runners, and all UI rendering logic built with Ink. This component handles message display, syntax highlighting, keyboard shortcuts, theming, slash command handling, authentication flows, and privacy consent screens. It also contains CLI-side services for command loading and prompt preprocessing.

In our conceptual architecture, we envisioned the Terminal Interface as a thin client that captures user input, forwards it to the Orchestration Engine, and renders responses. The concrete architecture reveals that Terminal has several additional contact points with other server-side components. It imports tool type definitions from the Toolbox to render confirmation dialogs, reads policy types from Security and Execution to adjust behavior based on approval modes, and contains slash command handlers that reach into the Extension Manager for features like `/extensions install`. These appear to be pragmatic

shortcuts where the CLI layer accesses server-side modules directly for display-related information, rather than routing everything through the Orchestration Engine as our layered model would prescribe.

3.1.2 Orchestration Engine

The Orchestration Engine is the central coordinator on the server side, driving the ReAct (Reasoning and Acting) loop for multi-step autonomous task completion. It receives requests from the Terminal, gathers context from the Configuration System, constructs prompts with tool schemas, sends them to the remote Gemini model (handling all API communication internally), and interprets responses to determine whether to display text, execute a tool, or continue reasoning. It also owns the confirmation bus, an event-based messaging system that requests user confirmation from the Terminal without importing UI code directly, which is why the concrete diagram shows no dependency from Orchestration to Terminal, cleanly validating our client-server boundary.

What we did not fully appreciate in A1 was the Orchestration Engine’s relationship with the Task Delegation System, which was absent from our original conceptual architecture. The Orchestration Engine coordinates three-way delegation: to the Toolbox for built-in tools, to the Extension Manager for MCP-provided tools, and to the Task Delegation System for agent-level task offloading. It also handles model fallback when rate limits are hit, telemetry tracking, lifecycle hooks around LLM requests, and Git-based checkpointing for session recovery.

3.1.3 Toolbox (Built-in Tools)

The Toolbox contains all built-in tool implementations and the skill system. Each tool (file reading, writing, shell execution, web fetching, code editing) follows a standardized interface with parameter validation, confirmation checks, and execution logic. The tool registry maintains a unified catalog and exposes tool schemas to the Gemini model. The skill system, accessed through `activate_skill`, loads specialized procedural expertise on demand, with only skill metadata discovered at session start to save tokens.

The Toolbox receives execution requests from the Orchestration Engine and consults Security and Execution for permission checks before performing operations. One aspect we did not anticipate was how tightly it interacts with the Extension Manager: MCP server tools register in the same tool registry as built-in tools, creating closer coupling between these two components than our conceptual model of independent peers suggested.

3.1.4 Extension Manager

The Extension Manager handles capabilities beyond built-in tools by managing MCP (Model Context Protocol) server integrations and third-party extensions. It discovers MCP servers from configuration, establishes connections, retrieves tool schemas, and coordinates execution of extension-provided tools. It also manages the extension lifecycle: installing, updating, enabling, disabling, and handling consent flows. The `third_party/` directory contains bundled dependencies like `get-ripgrep` for fast file searching. The Extension Manager enables the system to grow without modifying core code, aligning with our conceptual architecture’s emphasis on extensibility and the open-closed principle.

3.1.5 Security and Execution

Security and Execution enforces access control and provides execution isolation. The policy engine in `packages/core/src/policy/` evaluates each tool call against configurable TOML rules, returning one of three decisions: allow, deny, or `ask_user`. Rules are organized in three priority tiers: administrator rules for enterprise enforcement, user rules for personal preferences, and defaults where read operations are allowed while write operations require confirmation. A separate safety checking layer in `packages/core/src/safety/` provides additional validation beyond rule-based policies. On the execution isolation side, sandbox functionality manages container-based sandboxing through Docker, Podman, or LXC, as well as macOS Seatbelt profiles. Together, the policy engine and sandbox implement defense-in-depth: the policy engine decides *whether* an operation proceeds, while the sandbox controls *how* it

executes.

3.1.6 Task Delegation System

The Task Delegation System handles Agent-to-Agent (A2A) communication, enabling Gemini CLI to delegate complex tasks to remote specialized agents. This component was missing from our A1 report and was added after global feedback from Dr. Adams. The `packages/a2a-server/` package implements the server-side A2A protocol, while `packages/core/src/agents/` provides client-side logic for discovering agents and exposing delegation as a tool the model can invoke.

This component matters to the architecture because it enables distributed processing where specialized tasks can be offloaded to purpose-built agents. It supports both remote execution through A2A servers and local execution within the same process. It also represents a fundamentally different extensibility mechanism from the Extension Manager: while MCP extensions add individual tools, agent delegation hands off entire multi-step workflows to an agent that reasons independently.

3.1.7 Configuration System

The Configuration System manages all persistent state, settings hierarchies, and context injection. In our original conceptual architecture we called this the Memory Manager and envisioned it mainly handling conversation context and user preferences. Examining the codebase revealed the scope is far broader: configuration logic appears in `packages/core/src/config/` (the core `Config` class), `packages/cli/src/config/` (argument parsing and settings merging), and `scripts/` (build automation and schema generators).

The settings system implements a hierarchical merge strategy where project-level settings override user-level, which override system defaults. Beyond key-value configuration, this component loads `GEMINI.md` context files, manages conversation history compression when the token window fills up, and provides configuration-driven behavior to every other component. This cross-cutting nature is why we positioned it as spanning all server-side layers. The concrete dependency diagram confirms this: Configuration System has bidirectional dependencies with every other component.

3.2 Concurrency

In the source code, we were able to discover and understand the underlying mechanisms behind the system's concurrency and control flow. There were some differences between the concrete implementation and the conceptual architecture, particularly regarding the ReAct Loop implemented in `packages/core/src/agents/agent-scheduler.ts`. This file implements the action and observation part of the ReAct loop and manages the scheduler's interaction with other subsystems. Within the scheduler, `scheduleAgentTool` is used to execute tool call requests from the orchestration engine. It then creates a new scheduler instance by passing in a message bus, which allows other components to communicate asynchronously instead of using hierarchical calls. Concurrently, this step creates a proxy config that overrides the global tool registry by registering a new agent-specific tool that is accessible only to that agent. The orchestration layer can then cancel resource-intensive background processes immediately as the scheduler offloads large-scale system searches to the worker threads. This ensures the Node.js async loop, which is responsible for handling async operations, remains free to process incoming user interruptions from the terminal interface. After the worker thread completes its designated task, the orchestration layer packages the completed tool call and forwards it to the model's MCP, which is part of the task delegation system.

4 Reflexion Analysis: Only Unexpected Dependencies

During our reflexion analysis, we compared the concrete dependency diagram from Understand [1] against our updated conceptual architecture. The concrete architecture largely validates our conceptual design, but reveals several unexpected dependencies not anticipated in our conceptual model. We found no absent dependencies: every connection we predicted exists in the code.

To investigate the divergences, we used two approaches depending on the scale of the dependency. For

large dependency counts (dozens or hundreds of references), we examined the folder and file names involved in Understand’s dependency browser, since high-volume dependencies usually exist for valid architectural reasons such as implementing user-facing features or satisfying interface contracts. For single-digit dependencies, we drilled into the specific files using Understand’s dependency browser to inspect the exact import statements and determine whether the dependency reflects a genuine architectural relationship or is merely a type-level import that could be refactored into a shared module to eliminate the cross-component coupling.

Configuration System → all components (237, 18, 215, 12, 60, 24): Our conceptual architecture positioned the Configuration System as cross-cutting, with every component depending on it for settings, context, and session state. The concrete architecture confirms these inward dependencies but also reveals the reverse: Config System itself depends on every other component. This was not in our conceptual model, where arrows only pointed toward Configuration. In practice, the core `Config` class imports types and interfaces from other components to wire them together during initialization. For example, it imports tool registry types from the Toolbox, policy types from Security, and agent types from Task Delegation in order to construct the unified configuration object that it then provides back to everyone. This makes Configuration System not just a passive data store but an active integration point that needs to know about the components it serves.

Terminal Interface → Toolbox (864), Security and Execution (452), Extension Manager (180): Our conceptual architecture shows Terminal communicating only with the Orchestration Engine. The concrete code reveals three additional strong dependencies. Terminal imports tool types and the `ToolRegistry` interface from the Toolbox to render confirmation dialogs and display tool results. It imports policy types like `ApprovalMode` from Security to adjust UI behavior based on approval modes and sandbox status. And it contains slash command handlers for extension management (`/extensions install`, `/mcp add`) that import extension types. These are pragmatic shortcuts where the CLI reaches directly into server-side modules for display and command-handling purposes rather than routing through Orchestration.

Terminal Interface ↔ Task Delegation System (1/4): The single reference from Terminal to Task Delegation is a type-only import of `AgentDefinition` for displaying agent information. The 4 reverse references come from a2a-server command files importing `CommandContext` to register agent commands in the CLI. Both directions are type or registration imports with no behavioral coupling and could have been **eliminated**. The developers could extract these shared types into a common module to eliminate the cross-component dependency.

Security and Execution → Terminal Interface (38): Our conceptual architecture positions Security as a lower-layer component depended upon by others, not depending on the presentation layer. The 38 references stem from security configuration files we assigned to Security (`sandboxConfig.ts`, `policy.ts`, `trustedFolders.ts`) that import CLI-side utilities for settings loading. This is partly an artifact of our file-to-component mapping, where CLI-side security configuration naturally imports from both security logic and CLI infrastructure.

Upward dependencies to Orchestration Engine: Toolbox (164), Security (65), Task Delegation (103): Our conceptual architecture predicts downward dependencies from Orchestration to these three components but not the reverse. The concrete code shows substantial upward dependencies from all three. The Toolbox imports orchestration types because tool execution results must conform to interfaces defined in the core orchestration loop. Security imports session context and tool scheduling types from the Orchestration layer to evaluate policies accurately. Task Delegation imports core types because locally running subagents reuse the same orchestration infrastructure as the main session. These upward dependencies partially violate our layered model but are common in practice when lower layers need interface contracts defined by higher layers.

Extension Manager ↔ Toolbox (139/37): Our conceptual architecture treats these as independent peers. In practice, MCP server tools register in the same **ToolRegistry** as built-in tools and conform to the same **BaseTool** interface, because the Gemini model sees all tools as a flat catalog regardless of origin. This coupling is architecturally necessary for providing a unified tool interface to the model.

Task Delegation System → Toolbox (189): This was not predicted in our conceptual architecture. Agent delegation is exposed as a tool (`delegate-to-agent-tool.ts`) registered in the Toolbox’s registry, and subagents running locally need full access to the tool system to execute tools on behalf of delegated tasks. A locally running subagent is essentially a mini orchestration loop using the same Toolbox as the main session.

Security and Execution → Toolbox (219): Our conceptual architecture predicts the reverse direction (Toolbox consulting Security for permission checks), but the concrete code also shows a strong dependency from Security to Toolbox. The policy engine needs to inspect tool names, parameter schemas, and command prefixes to evaluate rules, which requires importing tool type definitions. This bidirectional coupling reflects the tight relationship between policy evaluation and tool execution.

Minor divergences: Security → Extension (3), Toolbox → Task Delegation (1), Extension → Orchestration (36), Extension → Terminal (9): We investigated the single-digit dependencies individually. The 3 references from Security to Extension come from `skillUtils.ts` reusing the Extension Manager’s Git cloning logic for skill installation. The single reference from Toolbox to Task Delegation is `tools.ts` importing `ToolConfirmationResponse` from the confirmation bus types near the agent code, a file-layout artifact. Both could be eliminated by refactoring into shared modules. The 36 references from Extension to Orchestration and 9 from Extension to Terminal exist because the extension system imports core types for session context and CLI types for command integration respectively, which are expected consequences of extensions needing to interoperate with the rest of the system.

5 Subsystem Analysis: Toolbox

The chosen subsystem that will be analyzed is the Toolbox; which is a major component of CLI architecture. The Toolbox subsystem is located within the Execution Logic Layer of Gemini CLI and is executed by the Orchestration Engine whenever the model may require to use built-in tools. The approach to developing the conceptual architecture for the subsystem started with analyzing the role of the Toolbox within the broader Gemini CLI conceptual architecture and identifying the key responsibilities this subsystem fulfills when invoked by other components. The conceptual architecture for the Toolbox is designed to focus on the logical process oriented subsystem required to support the execution of tasks invoked by the Orchestration Engine.

5.1 Conceptual and Concrete Architectures

The Conceptual architecture for the Toolbox is composed of 4 major **sub-subsystems**, each representing a specific responsibility within the workflow and showcasing dependency for proper task execution. The 4 components that make up the subsystems are: The **Tool Registry** acts as a centralized catalog that maintains information about all available tools in the system and enables other components to discover and access them. A **Skill** functions as the orchestration component that evaluates the assigned task and determines which tools should be used by referencing the **Tool Registry**. It coordinates the sequence in which these components are executed to accomplish the task. **Tools** are higher-level modules that perform complex operations; they receive instructions from the Skill component and execute the required tasks, delegating simpler operations to utilities when needed. **Utilities** provide lightweight helper functions that perform simple, low-level operations to support tools, allowing tools to execute tasks more efficiently by offloading basic functionality. Together, these components form a hierarchical workflow in which the Skill orchestrates the process, Tools handle complex execution, Utilities perform basic operations, and the Tool Registry maintains information about all available tools within the Toolbox subsystem.

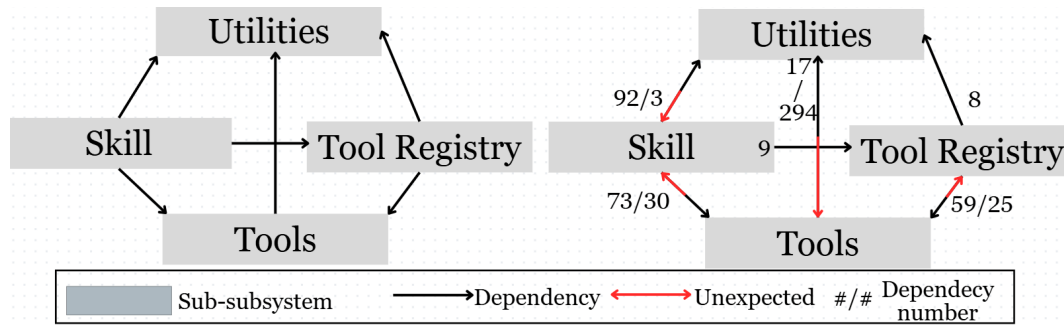


Figure 3: Conceptual and Concrete Architecture: Subsystem (Toolbox)

The main components mentioned here represent the subsystem at an abstract level and represent responsibilities that each component has to complete the subsystem. Furthermore, moving onto the concrete architecture; we start by mapping each file assigned to Toolbox in the broad concrete architecture to a specific component in the subsystem conceptual architecture.

5.2 Sub-subsystems Within Toolbox: Concrete

This section is to address the concrete architecture of the Toolbox subsystem. These component that are mention in the conceptual are also further understood in concrete design. This is done through Understand [1] and mapping which files reference to different components, and each of these groups represents a specific role within the execution workflow.

5.2.1 Skill

Skill provides task coordination within the Toolbox. Files such as `skillManager.ts`, `skillLoader.ts`, and `activate-skill.ts` manage the discovery, loading, and activation of tools. Skills analyze a task given by Orchestration engine, select appropriate tools through the Tool Registry, and orchestrate the sequence of tool executions needed to complete the task.

5.2.2 Tool Registry

Tool Registry is implemented through files like `tool-registry.ts` and `tool-names.ts`. It contains mappings between tool names and their implementations. The registry stores metadata describing the capabilities of each tool, expected parameters, and invocation process. This kind of properties of Tool Registry made possible that Skill can use the tool regardless how they are implemented.

5.2.3 Tools

Tools contains files that are responsible for executing specific tasks. Tools contain files like `read-file.ts`, `write-file.ts`, `grep.ts`, `glob.ts`, `web-search.ts`, and `shell.ts`. Each is in charge of a specific task such as interacting with the file system, executing shell commands, searching project content, or retrieving external information. Tools are invoked by the Skill component through the Tool Registry to perform requested operation and returning structured results. There are also files such as `edit.ts`, `pathReader.ts`, and `memoryTool.ts`, which handle more specialized operations like code editing, path resolution, and persistent memory management.

5.2.4 Utilities

Utilities provide low-level helper functions to upper-level component like Tools. Files such as `fileUtils.ts`, `debugLogger.ts`, `schemaValidator.ts`, `workspaceContext.ts`, and `errorReporting.ts` implement reusable functionality including file handling, logging, validation, environment context management, and error processing. Utilities collect these functionalities within one sub-subsystem so that other sub-subsystems do not need to implement them repeatedly, avoiding code duplication.

5.3 Reflexion Analysis for Toolbox: Unexpected Dependencies

The reflexion analysis is meant to compare the conceptual architecture of the Toolbox subsystem with the concrete architecture. The purpose is to identify the difference between the conceptual design to the actual dependencies present in the code base. Additionally, it would be ideal for the implementation to follow the concrete workflow defined in the conceptual architecture; where the Skill component manages highest level implementation, Tools execute specific tasks, Utilities provide low level helper functions, and the Tools Registry maintains information about tools. The concrete architecture fully supports the conceptual framework, but there several dependencies that are different from the conceptual design.

Utilities, Tools → Skill (3/30): The Skill component is responsible for coordinating execution and management of skill execution for any given task and it is the highest level component in our system. Dependencies are expected to go from Skill towards Tools and Tool Registry due to the given responsibilities. However, during reflexion analysis we got 3 from Utils and 30 from Tools pointing back to Skills. This suggests that some utility or tool function import Skill related functionality directly, which was not predicted in the conceptual arch. For example, tools may import message or event definitions used to communicate execution results back to the Skill component, or reference shared data structures related to skill activation and execution state. In some cases, utility functions may also reference skill configuration or helper functions that are implemented within the Skill module. These shared references result in a dependency on the Skill component, since it is the highest level executioner.

Utilities → Tools (17): The strong dependency from Tools to Utilities (294) is entirely expected, as every built-in tool relies on shared utilities for error handling, logging, file path resolution, and workspace context. The 17 reverse references from Utilities to Tools are less intuitive, since a low-level utility layer should not typically need knowledge of the higher-level components it serves. Investigating in Understand[1], we found that these references come from utility modules like `workspaceContext.ts` and `toolCallContext.ts` that import tool type definitions in order to provide context objects shaped around the tool execution pipeline. Rather than being pure low-level helpers, these utility files are context-aware services designed specifically to support tool execution, which requires them to understand the interfaces of the tools they serve. This is a reasonable trade-off that keeps shared context logic centralized in Utilities rather than duplicated across individual tool files.

Tools → Tool Registry (59): At first, in the conceptual architecture of the Toolbox, the Tool Registry was defined as the component responsible for keeping track of all available Tools, which suggests that the Tool Registry depends on Tools. After reviewing the files assigned to the Tool Registry, we observed a deeper connection between Tools and the Tool Registry. This connection shows Tools is depended on Tool registry, however, this does not imply that the execution of Tools depends on the Tool Registry. Rather, Tools import certain elements from the Tool Registry, such as `READ_FILE_TOOL_NAME` to provide a consistent, canonical name when registering themselves into Tool registry. This ensures that Skill can discover and invoke each Tool properly from Tool registry with the name.

6 External Interfaces

6.1 System Terminal

The terminal environment acts as the primary external interface for Gemini CLI, which runs as a process that communicates through standard input and output streams. These streams serve as the communication channel between the system and the user, allowing the CLI to receive commands and display responses. This interaction is initialized in `packages/cli/src/gemini.tsx`, which launches the connection to the terminal session. The source code also has UI components that handle terminal based user interaction and display logic which could be found in `packages/cli/src/ui/`. This includes files such as `AppContainer.tsx`, which manage how prompts and responses are rendered within the terminal. Because all user input and output flows through the terminal, it functions as a primary external interface for Gemini CLI.

6.2 External AI Model Interface (Gemini LLM)

Gemini CLI uses an external AI interface to send user prompts to the Gemini LLM through a cloud API. In the concrete architecture, this functionality is distributed across multiple components that handle request orchestration, authentication, and other related responsibilities, rather than relying on a single component. The main interaction with the Gemini service occurs in `packages/core/src/core/client.ts`, which acts as the service client responsible for managing communication with the Gemini model. Rather than following a simple request/response pattern, in which a complete prompt is sent and a full response is returned, the client uses message streaming to continuously manage communication with the model. This allows responses to be received incrementally and in coordination with the ongoing conversation. The authentication functionality is handled separately in `packages/cli/src/core/auth.ts`, which manages the initial authentication flow and validates credentials required to authorize requests to Gemini's remote services.

6.3 External File System Interaction

The local file system functions as an external interface for Gemini CLI, allowing the system to access and modify local files stored in the user's environment. Gemini CLI interacts with the file system to read source code, documents, and configuration files so their contents can be incorporated into AI prompts. These capabilities are implemented through a tool based architecture within the codebase. The relevant implementations are located in `packages/core/src/tools/`, which contains files such as `read-file.ts` and `write-file.ts` that retrieve and modify file contents, convert file data into structured inputs for the Gemini model, and manage outputs written back to the file system. The same module also includes file discovery and search utilities such as `glob.ts` and `grep.ts`, which allow the system to locate files and search through project contents.

6.4 Networking and Communication Interfaces

Networking and communication function as an external interface for Gemini CLI, allowing the system to interact with remote services such as the Gemini LLM. Communication with the Gemini model occurs through HTTPS based API requests sent to Gemini's cloud services. This interaction is coordinated by the service client layer located in `packages/core/src/core/client.ts`, which manages chat state, conversation turns, and tool availability during communication with the Gemini model. In addition to model communication, Gemini CLI supports integration with external tools through the Model Context Protocol. The MCP implementation is primarily located in `packages/core/src/tools/mcp-client.ts`, which allows the CLI to connect to external MCP servers and access additional tools and resources outside the system. Networking also includes Gemini CLI's Agent-to-Agent server interface, which allows external systems to interact with the CLI through A2A based requests. This functionality is implemented in `packages/a2a-server`, with agent client management defined in `packages/a2a-server/src/a2aClientManager.ts` and task execution logic in `packages/a2a-server/src/agent/executor.ts`.

6.5 Integration with External Tools and Workflows

Integration with external command line workflows also serves as an external interface for Gemini CLI. Because the system operates within a standard terminal environment, it can accept input from other programs and return output that can be redirected or piped into other tools. This allows Gemini CLI to easily integrate into broader development workflows such as shell scripting, build pipelines, and code analysis. Rather than relying on a dedicated workflow integration component, this capability comes naturally from the CLI architecture itself. This behavior is initiated through `packages/cli/src/gemini.tsx`, which launches the command line process. Once it is running, the system can receive input streams from the shell and return generated output through the same channels.

7 Use Cases

7.1 Use Case 1: Summarize A Stock

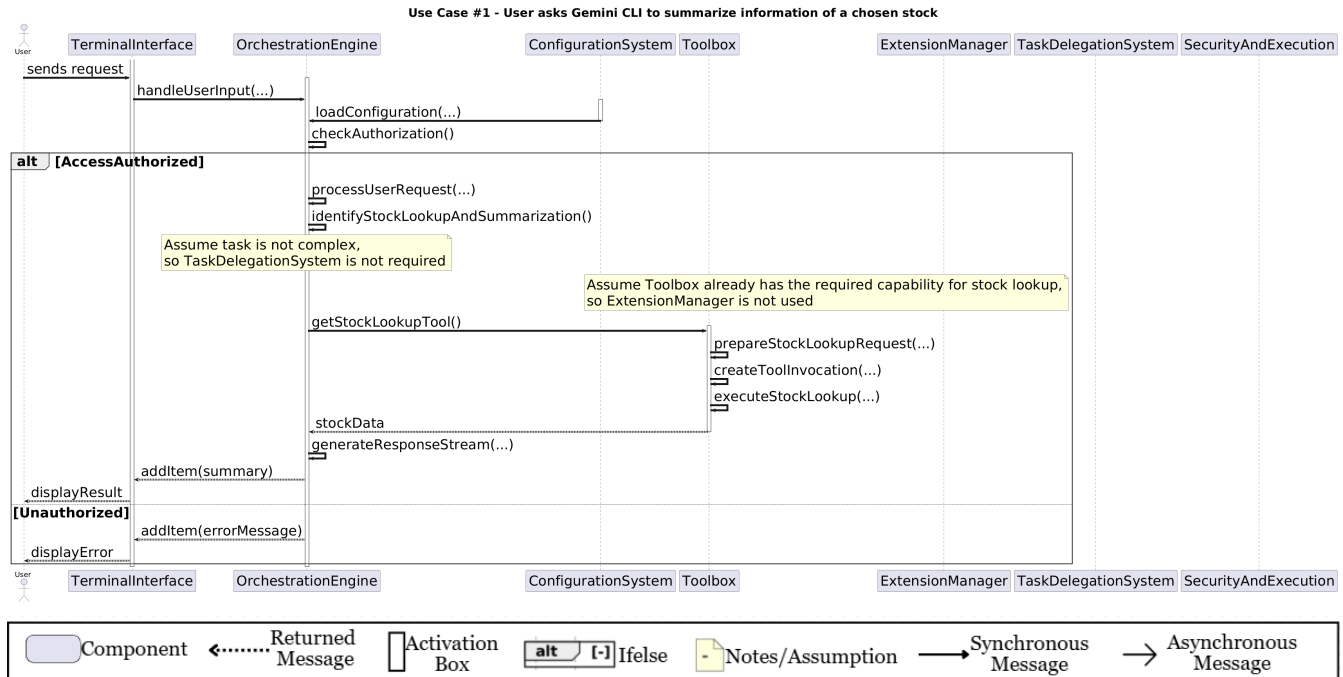


Figure 4: User asks Gemini CLI to summarize information of a chosen stock

Before describing the sequence flow for this use case, it should be noted that several component adjustments were made to the diagram based on the concrete architecture, with components being merged, expanded, and reinterpreted to better reflect the actual structure of the system. The AgentCoreCoordinator and APIMessenger, which were initially modeled as separate components for workflow coordination and Gemini communication, were merged into a single OrchestrationEngine since the concrete implementation handles these responsibilities within one coordinating layer. The TaskDelegationSystem was also introduced after finding out that complex tasks are delegated outside the main orchestration logic. In the updated design, the OrchestrationEngine delegates these complex tasks to this component, which operates in parallel with the Toolbox and ExtensionManager. Additionally, the PolicyManager and RiskPrevention components were merged into a single SecurityAndExecution component to more accurately represent the concrete architecture. This component acts as a unified security layer responsible for safety checks, policy validation, and sandbox simulations. Finally, the MemoryManager was replaced with a ConfigurationSystem for contextual data, and the UI component was updated to TerminalInterface to better reflect Gemini CLI's command line interaction model.

The diagram begins when the user submits a request for a stock summary through the TerminalInterface. This request is passed to the OrchestrationEngine through `handleUserInput(...)`, which is the point where the system receives and begins processing the user's input. The OrchestrationEngine then interacts with the ConfigurationSystem through `loadConfiguration(...)` to access the current configuration and context needed for the request. Before any further processing occurs, the OrchestrationEngine checks authorization to Gemini services by calling `getAuthorizationStatus()` on itself. Since the APIMessenger's functionality is now integrated into the OrchestrationEngine, authorization checks can be handled directly within this component. If authorization fails, the method call returns unauthorized, and the OrchestrationEngine sends an error message back to the TerminalInterface. If authorization succeeds, the OrchestrationEngine proceeds with `processUserRequest(...)` and `identifyStockLookupAndSummarization()`, which handle the incoming request and determine the necessary operations. At this point, the diagram has already diverged significantly from the conceptual architecture. Rather than relying on multiple coordinating components

like the ACC, APIMessenger, and ToolScheduler; the OrchestrationEngine manages the flow internally in a more direct and streamlined manner. After the request has been processed and interpreted, the OrchestrationEngine calls `getStockLookupTool()` on the Toolbox to obtain the appropriate stock lookup capability. This is because the stock lookup task is assumed to be simple, meaning the TaskDelegationSystem is not needed. Similarly, since the required capability is already assumed to exist in the Toolbox, the ExtensionManager is not involved either. Within the Toolbox component, `prepareStockLookupRequest(...)`, `createToolInvocation(...)`, and `executeStockLookup(...)` occur. These method calls reflect the concrete architecture's tool execution flow more accurately than the conceptual diagram did. Once the stock data is retrieved, it is returned to the OrchestrationEngine. Finally, the OrchestrationEngine calls `generateResponseStream(...)` to produce the final summary based on the retrieved stock information. The resulting summary is then returned to the TerminalInterface through `addItem(summary)`, which is then displayed to the user. Compared to the conceptual model, this updated sequence is more accurate, direct and streamlined. It also emphasizes how the concrete implementation handles authorization, tool retrieval, execution, and response generation with fewer intermediate coordination steps.

7.2 Use Case 2: Translate a File From English to French

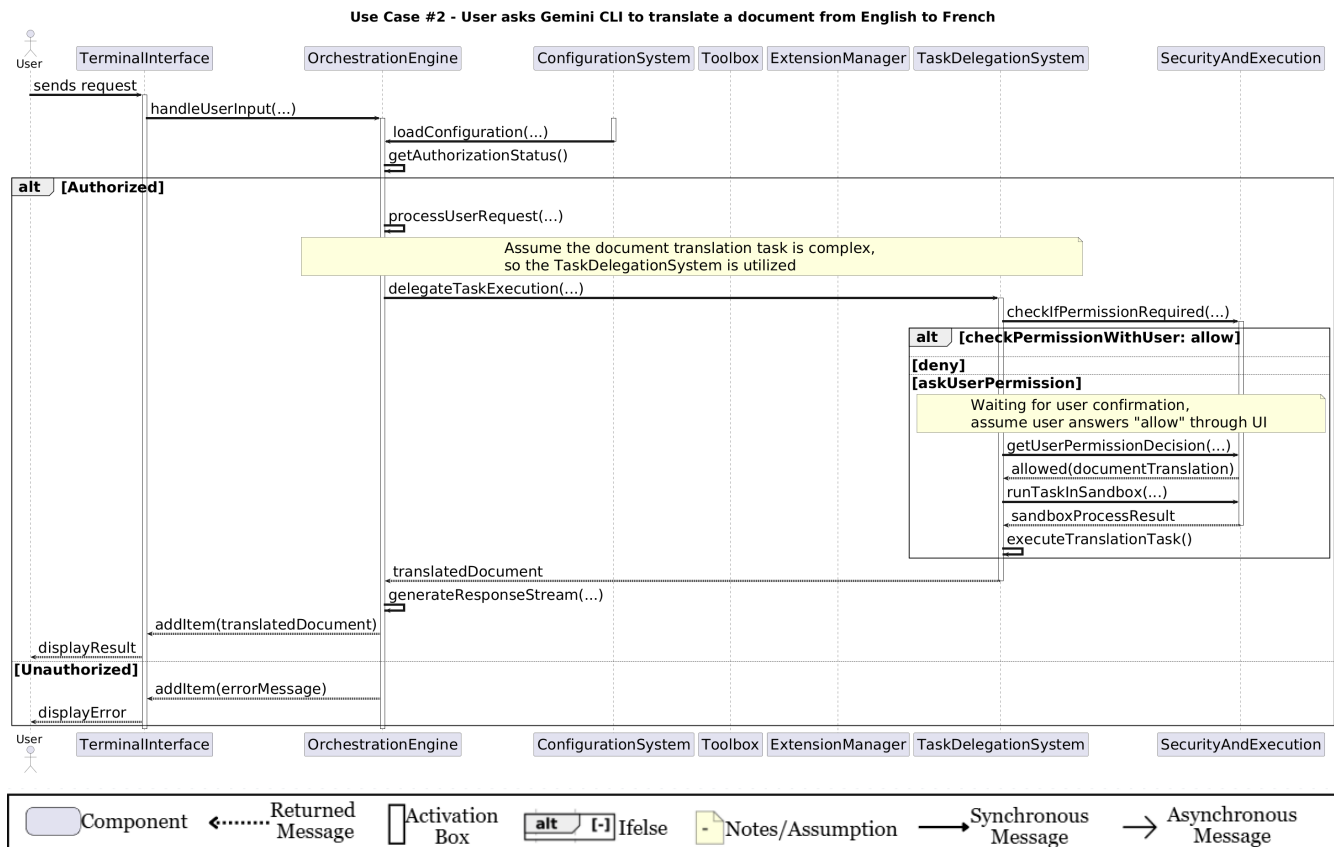


Figure 5: User asks Gemini CLI to translate a file from English to French (potentially dangerous action)

In the conceptual architecture report, Use Case 2 originally involved deleting a file; this was replaced with translating a document from English to French. This change was intentionally made to increase the complexity of the task in order to demonstrate the behavior of the newly introduced TaskDelegationSystem, a crucial component of the system architecture. Nonetheless, this use case still preserves the core idea of modifying a file, which represents a potentially dangerous operation.

Up until the authorization step, this sequence diagram works exactly like Use Case 1. The new sequence events occur after the authorization status is returned. If authorization fails, `getAuthorizationStatus()` returns unauthorized, and the OrchestrationEngine sends an error message back to the TerminalInterface.

face. If authorization succeeds, the `OrchestrationEngine` continues with `processUserRequest(...)`, which interprets the request and determines how the task should be executed. Unlike Use Case 1, the task is assumed to be a complex operation. As a result, the `OrchestrationEngine` delegates it to the `TaskDelegationSystem` through `delegateTaskExecution(...)`. This reflects a key aspect of the concrete architecture, where complex tasks are handled outside the main orchestration layer. Once the `TaskDelegationSystem` receives the request, it first consults `SecurityAndExecution` through `checkIfPermissionRequired(...)` to determine whether the operation requires additional user permission. Since translating a document involves modifying file content, it is treated as a potentially dangerous operation. The system then waits for user input and retrieves the decision through `getUserPermissionDecision(...)`. If permission is denied, the process terminates; otherwise, the operation proceeds. After permission is granted, `SecurityAndExecution` returns the user decision, and the `TaskDelegationSystem` requests a secure execution environment through `runTaskInSandbox(...)`. The sandbox simulation then returns `sandboxProcessResult`, confirming that the task can safely proceed. The `TaskDelegationSystem` then performs the translation through `executeTranslationTask()`, after which the translated document is returned to the `OrchestrationEngine`. The `OrchestrationEngine` then calls `generateResponseStream(...)` and sends the result to the `TerminalInterface` through `addItem(translatedDocument)`, where it is displayed to the user. Compared to the conceptual architecture, the main difference in this sequence is the role of the `TaskDelegationSystem`, which more accurately reflects how this task would be handled in the concrete implementation.

8 Lessons Learned

Recovering the concrete architecture of a large system like Gemini CLI was a considerably different challenge from writing the conceptual architecture. Many of the difficulties we encountered were ones we did not anticipate, and the lessons below reflect what we would do differently next time.

The most significant lesson was that concrete architecture recovery cannot be parallelized the way report writing can. We assumed that since everyone understood the conceptual architecture from A1, we could divide the Understand mapping and work independently. In practice, deciding which folders belong to which component and how to handle boundary files required collective decision-making at almost every step, since changing one file's assignment could shift dependency counts across the entire diagram. We went through several rounds of remapping before arriving at a diagram we trusted, and only started writing less than a week before the deadline. We should have treated the Understand mapping as the critical-path task from day one.

Working with Understand [1] and the Gemini CLI codebase [3] also taught us about good developer practices. Understand is powerful but overwhelming on a large codebase, and distinguishing meaningful architectural dependencies from incidental type imports required patience and systematic investigation. The Gemini CLI codebase made this harder because commit messages and comments were often vague, making it difficult to determine the developers' intentions behind certain structural choices. This reinforced that writing clear commits and thorough comments matters not just for one's own team but for anyone who needs to understand the architecture later.

Finally, we learned to keep a mental roadmap when navigating a system at this scale. The approach that worked best was to start from the conceptual architecture as a frame of reference, then investigate discrepancies rather than trying to understand every file from scratch. Having the conceptual model as a baseline made unexpected dependencies something concrete to investigate against, which was far more productive than exploring without a hypothesis.

9 Conclusion

This report presented a recovery of the concrete architecture of the Gemini CLI system through analysis of its source code and dependency structure. Using the Understand static analysis tool, we extracted dependency data and organized system components into major subsystems to better understand how the

system is implemented in practice. The recovered architecture revealed a system centered around an orchestration engine that coordinates model interactions, tool execution, configuration management, and security mechanisms through several supporting subsystems.

Through a reflexion analysis comparing the recovered architecture with the conceptual architecture proposed in Assignment 1, we identified both similarities and notable differences between the intended design and the implemented structure. While many aspects of the conceptual architecture were validated, the concrete implementation revealed tighter coupling between certain components and responsibilities that cross architectural layers, reflecting practical design decisions made during development.

Overall, the concrete architecture provides a clearer understanding of how Gemini CLI operates internally and highlights important subsystem relationships that were not fully visible from documentation alone. These findings improve our architectural understanding of the system and provide a solid foundation for evaluating potential architectural improvements and proposing new features in the next stage of the project.

10 Dictionary and Naming Conventions

Term	Definition
CLI	(Command Line Interface) Text-based interface for system interaction via commands.
GEMINI.md	Per-project markdown context files loaded by the Configuration System to provide project-specific instructions and context to the AI model.
MCP	(Model Context Protocol) Standardized protocol connecting AI models to external data sources and tools.
ReAct Loop	(Reasoning and Acting Loop) The iterative cycle where the AI model reasons about a task, requests a tool action, observes the result, and repeats until the task is complete.
Sandbox	Isolated environment for executing code safely without affecting the host.
TOML	(Tom’s Obvious Minimal Language) Minimal configuration file format designed for readability.
Toolbox	The component containing all built-in tool implementations such as file operations, shell commands, and web access, as opposed to the Extension Manager which handles third-party tools.
Agent to Agent (A2A)	A communication interface or server that enables multiple autonomous AI agents to interact, delegate tasks, and share context directly with one another without requiring human intervention.
Unexpected Dependency	A dependency that exists in the code but is not allowed by the conceptual architecture.
Absent Dependency	A dependency existing in the conceptual architecture but not in the concrete one.

Table 1: Dictionary and Naming Conventions

11 AI Collaboration Report:

11.1 AI Member Profile and Selection Process:

For this assignment, we will continue to use Gemini 3 Pro (November 2025 version). In Assignment One, Gemini 3 Pro demonstrated a solid ability to help us understand the conceptual architecture. For Assignment Two, we are investigating the concrete architecture of the Gemini CLI; since the Gemini CLI and Gemini utilize the same infrastructure, it is logical to continue using this model.

11.2 Tasks Assigned to the AI Teammate:

- **Generate High Level Mapping of Dependencies:** We used this mapping of function calls and relationships to gain a preliminary understanding of the project's structural direction. Gemini was able to immediately categorize source files into their respective conceptual architectural components.
- **Paragraph(s) refinement:** The AI efficiently identified grammatical errors, spelling inconsistencies, and logical gaps within each paragraph that may have been overlooked during our manual proof-reading sessions.
- **Code Analysis for Architecture Mapping:** We used SciTools Understand to extract the concrete architecture by mapping raw source-code dependencies to specific components. We then utilized AI to quickly compare this implementation with our conceptual architecture, which significantly accelerated the overall mapping process.

11.3 Validation and Quality Control Procedures:

Similar to the Validation and Quality Control process section in the Conceptual Architecture report, a major setback when using an LLM is its tendency to hallucinate. To mitigate this, we implemented strategies such as verifying the credibility of the sources provided and cross-referencing all data from the LLM with independent research and web searches. A key development in this assignment was the use of the LLM to quickly check the CLI codebase, identifying dependencies and function calls between various subcomponents. To ensure accuracy, any results generated by the AI were manually confirmed by examining Git commit histories [3] and documentation, verifying that the identified dependencies were indeed present in the actual implementation.

11.4 Interaction Protocol and Prompting Strategy:

Building on our experience from Assignment One, we initially attempted to have each team member engage directly with the AI by creating their own prompts for their specific tasks. However, this time we implemented a peer-review step, requiring the group to reach a consensus on whether a prompt was complete before execution. Unfortunately, this approach led to significant disagreement and disorder within the team. As a result, we reverted to our previous strategy of appointing a single person to be in charge of prompt scheduling and management to ensure consistency and efficiency.

The prompting strategy we chose to implement for this assignment follows a structured three-step plan. First, we provide Gemini with the initial user query we wish to have addressed. Second, we utilize few-shot prompting by providing two to five examples of successful interactions; this allows the model, during its inference phase, to learn the structure and rationale behind those examples and replicate them while adhering to our specific constraints. Third, we implement a feedback loop to critically analyze the AI's output for accuracy, tone, and relevance. In this stage, we identify specific errors or areas where the model failed to meet our needs. If a lack of understanding is detected, we provide additional background information and refine the prompt to be more specific and aligned with our goals. This iterative process continues until the final output reaches a satisfactory standard.

Example prompt:

Initial Query: As a specialized software architecture professor, you will rigorously analyze and explain the

MCP's interactions with other components within the CLI's system architecture.

Context: The Model Context Protocol (MCP), mainly found in `packages/core/src/tools/mcp-client.ts` and overseen by the Extension Manager, offers a consistent way for the Gemini CLI to find and link with outside servers, which helps grow the system's list of tools and resources while keeping the core architecture flexible.

Task: Explain how the MCP and the Extension Manager interact with the rest of the seven subcomponents.

Example Query: Explain what the ReAct Loop does in `packages/core/src/scheduler/scheduler.ts` and how it's related to the seven subcomponents of our architecture.

Example Response: The Scheduler, implemented in `packages/core/src/scheduler/scheduler.ts`, orchestrates the Reasoning and Action phases by executing tool requests through an asynchronous event loop that maintains system responsiveness. This layer ensures domain isolation via agent-specific registries and returns results as a `CompletedToolCall` to the model for further reasoning cycle.

11.5 Reflection on Human-AI Team Dynamics:

Integrating AI into our concrete architecture report was an iterative process that we already developed during our conceptual architecture report. We did not require much time in experimenting with methods, as we were already familiar with how Gemini works. This prior experience allowed use to use the AI in a more strategic and efficient manner.

In the conceptual architecture report, we primarily used the model to brainstorm and generate ideas for the content we needed to write. In contrast, for the concrete architecture report, we used the AI mainly for analytical tasks, such as examining the Gemini CLI codebase, as well as minor tasks like reviewing written sections for clarity, grammar, spelling, and fluency. The AI was extremely beneficial in helping our team identify logic gaps and details that we might otherwise have overlooked.

Similarly to the first report, the AI's involvement introduced additional work, particularly in validating outputs and ensuring that the technical information about the concrete architecture was accurate. However, compared to our previous experience, this process was a lot more streamlined. This is because during the development of the first report, we had already established clear guidelines for when to use AI and how to use it appropriately. As a result, we did not waste any time on resolving disagreements during meetings or addressing ancillary tasks; this meant that the time saved by using the tool significantly outweighed the time lost to managing it.

Overall, this report reinforced our earlier learning that effective collaboration with AI relies on considerate use, evaluation of outputs, and clear team communication.

References

- [1] SciTools, *Understand: Static code analysis tool*, Software application, 2026. [Online]. Available: <https://scitools.com/>.
- [2] Dr. Bram Adams, *Gemini cli - conceptual architecture summary*, Available via onQ learning management system, Course document, PDF, 2026.
- [3] Google, *Gemini cli*, version 1.0, Nov. 18, 2025. [Online]. Available: <https://github.com/google-gemini/gemini-cli>.
- [4] D. Garlan and M. Shaw, “An introduction to software architecture,” Carnegie Mellon University, School of Computer Science, Tech. Rep. CMU-CS-94-166, 1994. [Online]. Available: https://userweb.cs.txstate.edu/~rp31/papers/intro_softarch.pdf.
- [5] Google Gemini Team. “Gemini cli architecture overview.” Official Gemini CLI Documentation, Accessed: Jan. 28, 2025. [Online]. Available: <https://geminicli.com/docs/architecture/>.